

Authentication Model

Sprint Education Website

Student & Admin Login — Design Overview

1. Objective

The Sprint education website needs secure login for two types of users: Students and Admins. The goal is an authentication system that is easy to use for students, strongly protected for admins, and simple to maintain and grow over time.

Any approach chosen needs to balance six things:

- Security — protecting accounts and data
- User experience — fast, frustration-free login
- Implementation complexity — realistic to build and test
- Scalability — works as the number of users grows
- Cost — reasonable to build and run
- Maintainability — easy for the team to support long-term

Key design principle: Students and Admins log in through the same central authentication system, but with different security rules applied to each — students get a simple, fast login, while admins get extra protection.

2. How Login Works

There are two separate login journeys, shown below. Both start the same way, but the Admin journey has one important extra step: mandatory multi-factor authentication (MFA).

2.1 Student Login Flow

Students get a quick, simple login — just an email and password. No extra steps are required, since a compromised student account carries lower risk than an admin account.

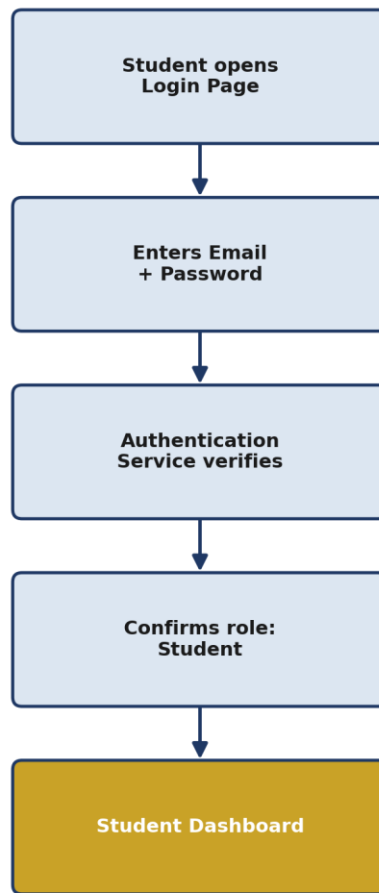


Figure 1 — Student login flow

2.2 Admin Login Flow

Admins go through the same first step (email + password), but must then pass a mandatory second factor — such as a one-time code — before the system checks their role and permissions and lets them in. This protects the most sensitive parts of the site.

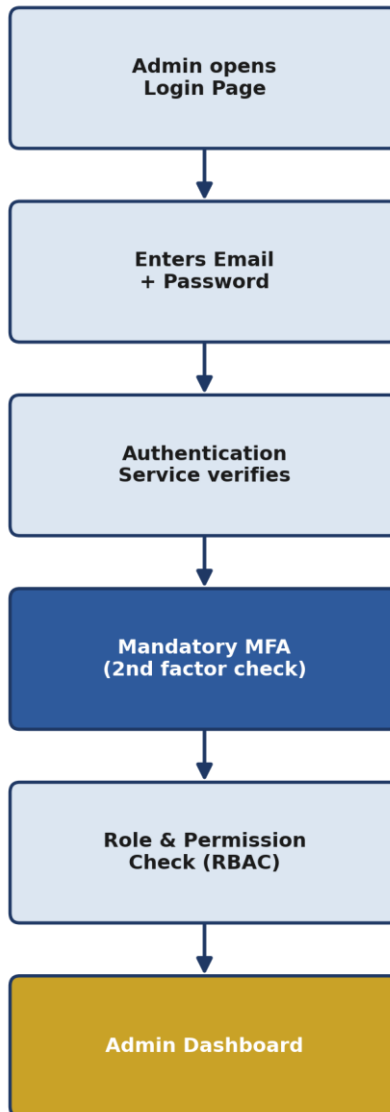


Figure 2 — Admin login flow (with mandatory MFA)

3. System Architecture

The diagram below shows how login requests move through the system: the React frontend collects credentials, the Node.js/Express backend runs the appropriate authentication check (student or admin), and once a user is verified, an authorization layer (role-based access control, or RBAC) decides what that user is allowed to see and do. All user data lives in MongoDB.

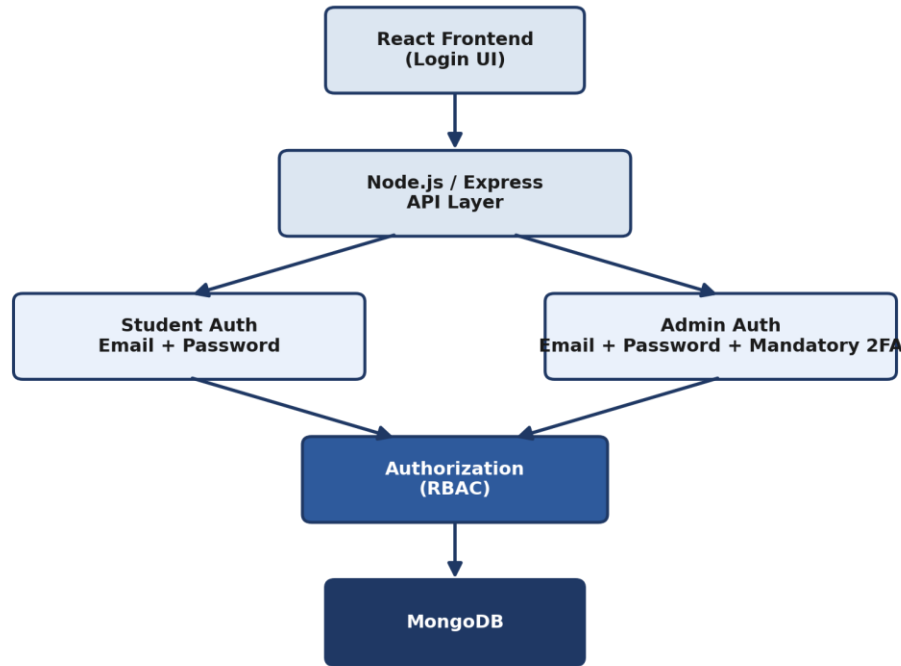


Figure 3 — End-to-end authentication architecture

4. Proposed Security Baseline

This table summarizes the recommended security level for each user type today, and leaves room to add new user types later without redesigning the system.

User Type	Authentication	MFA / 2FA
Student	Username/Email + Password	Not required
Admin	Username/Email + Password	Mandatory
Future user types	Extensible — reuses the same system	Can be added later

5. Implementation Approach Options

There are two broad ways to build this: a custom authentication system built into our own backend, or a third-party identity provider (such as Auth0 or Firebase Authentication) that handles login on our behalf. Both are viable for a MERN application; the table below compares them.

	Option 1 — Custom Authentication (Recommended)	Option 2 — Third-Party Identity Provider
What it is	Login, password storage, sessions/tokens, and MFA all built and hosted inside our own Node.js/Express backend.	A managed service (e.g., Auth0, Firebase Auth, AWS Cognito) handles login and hands our backend a verified token.
Full control	Full control over data, flows, and branding.	Less control — tied to the provider's features and roadmap.
Setup effort	More upfront development, but no ongoing per-user fees.	Faster to set up initially; usage-based pricing as the user base grows.
Data ownership	All user data stays in our own MongoDB database.	Some identity data lives with the third party.
Best fit	Best when we want full ownership of user data and predictable long-term cost — a good fit for an education platform handling student records.	Best for teams wanting to launch fast and outsource security upkeep.

Recommendation: Option 1 (Custom Authentication) is recommended, since it keeps full control of student and admin data in-house and avoids recurring per-user identity fees, while still being straightforward to build with the existing Node.js/Express/MongoDB stack.

6. Admin MFA — Method Options

For the mandatory second factor on admin accounts, three common methods were considered:

- Option A — Email/Username + Password only (no second factor) — simplest, but does not meet the mandatory MFA requirement for admins.
- Option B — Password + Email OTP — a one-time code is emailed at login; easy for admins to use, no extra app required.
- Option C — Password + Authenticator App (e.g., Google/Microsoft Authenticator) — strongest protection, since codes are generated on the admin's device and not sent over email.

Recommendation: Option C (Authenticator App) for the strongest protection, with Option B (Email OTP) as a simpler fallback recovery method if an admin loses access to their authenticator app.

7. Security Features Checklist

Regardless of which option is chosen, the following protections apply across the whole system:

- HTTPS everywhere — all traffic encrypted in transit
- Secure password hashing — passwords are never stored in plain text
- Secure, HttpOnly cookies or token-based sessions
- Rate limiting and login-attempt protection — blocks brute-force attempts
- Session management — safe login/logout and session expiry
- Self-service password reset
- MFA recovery process for admins who lose their second factor
- Audit logs — record of login activity, especially for admin accounts
- Ongoing admin security monitoring

8. Summary

Students get a fast, simple login. Admins get the same familiar login screen, but with a mandatory extra security check before they can reach sensitive admin tools. Both flows run through one shared, custom-built authentication system on our Node.js/Express backend, backed by MongoDB — keeping the system simple today, secure for admins, and easy to extend to new user types in the future.